

Comprehensive Project Report

Gemini

October 23, 2025

1 Introduction

In this report, we document the work done on this project. We will cover the entire pipeline, from data engineering and feature selection to predictive modeling and explainability. Our goal is to provide a clear and comprehensive overview of our methodology and findings.

2 Project Structure

The project is organized into several directories:

- `src/`: Contains the Python source code for data processing, modeling, and utilities.
- `dbs/`: Holds the datasets, divided into raw, intermediate, and cooked (processed) data.
- `jpy/`: Includes Jupyter notebooks used for exploration and analysis.
- `reports/`: Stores the output of our analysis, including explainability plots.

3 Data Engineering

We started by exploring the provided datasets to answer some key questions about flight bookings and traveler behavior. Our initial exploration was done in a Jupyter Notebook (`jpy/main.ipynb`).

3.1 Initial Data Exploration

We loaded four datasets:

- `AirlineScrappedReview_Cleaned.csv`
- `Customer_comment.csv`
- `Survey_data_Inflight Satisfaction Score.csv`
- `Passanger_booking_data.csv`

Our primary focus for the initial data engineering questions was on the `Passanger_booking_data.csv` dataset. We investigated the distribution of bookings, popular routes, and booking times.

3.2 Answering Key Questions

3.2.1 What are the most popular flight routes?

We analyzed the 'route' column to determine the most popular flight routes, both for all bookings and for verified bookings (where `booking_complete` is 1). We found that the top 10 routes differed slightly between the two groups. The analysis and visualizations can be found in our exploration notebook.

3.2.2 What is the distribution of bookings across flight hours?

We also analyzed the distribution of the number of passengers across different flight hours. We created histograms to visualize this distribution for both verified and all bookings. This helped us understand the peak booking times.

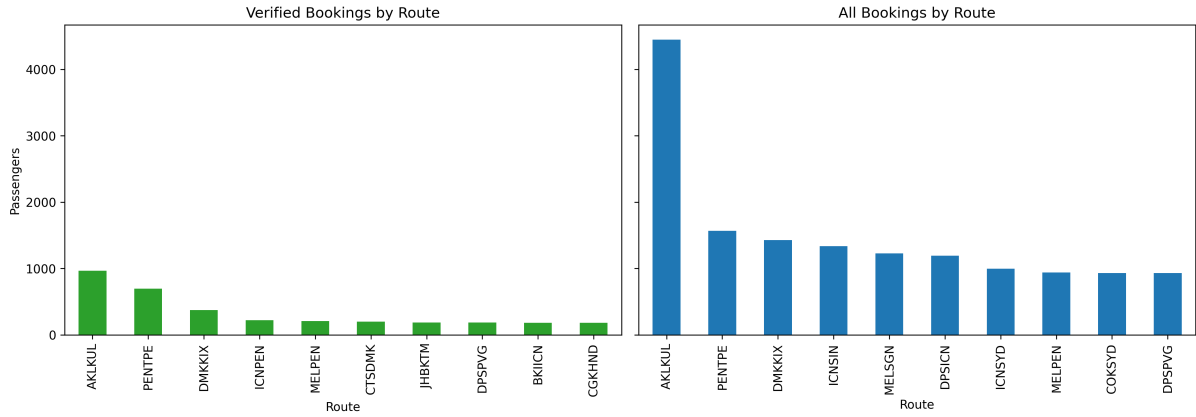


Figure 1: Top 10 flight routes for verified vs. all bookings (visualization from notebook).

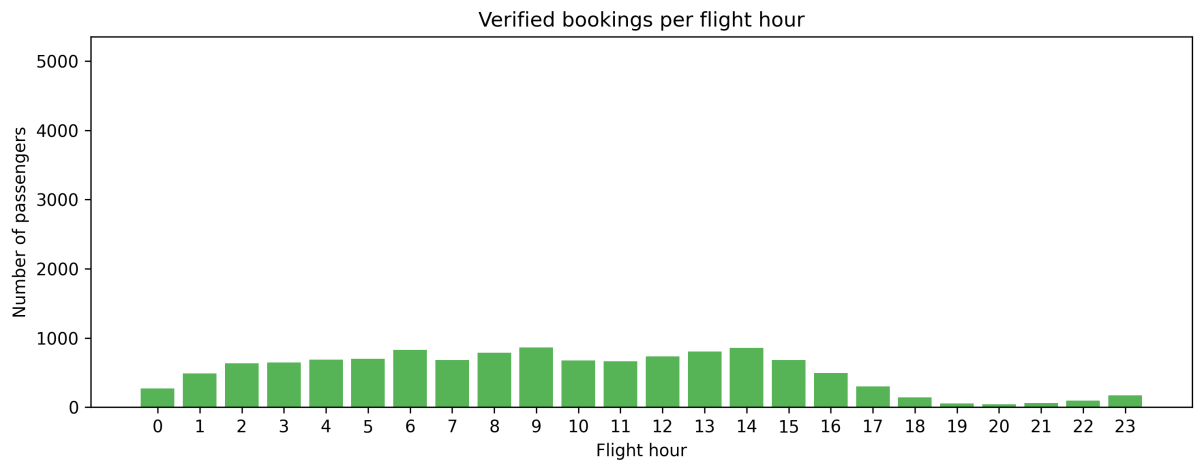


Figure 2: Distribution of verified bookings across flight hours (visualization from notebook).

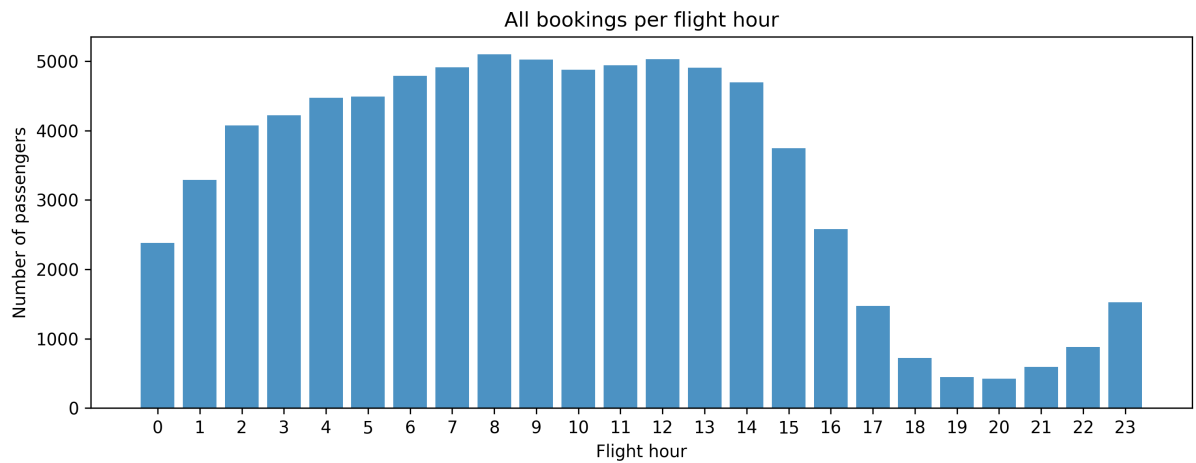


Figure 3: Distribution of all bookings across flight hours (visualization from notebook).

3.2.3 Which traveler type and class combination yields the highest and lowest ratings?

For this question, we used the `AirlineScrappedReview_Cleaned.csv` dataset. We grouped the data by 'Traveller_Type' and 'Class' and calculated the mean 'Rating'. We found that 'Solo Leisure' travelers in 'First Class' had the highest average rating, while 'Business' travelers in 'Economy Class' had the lowest.

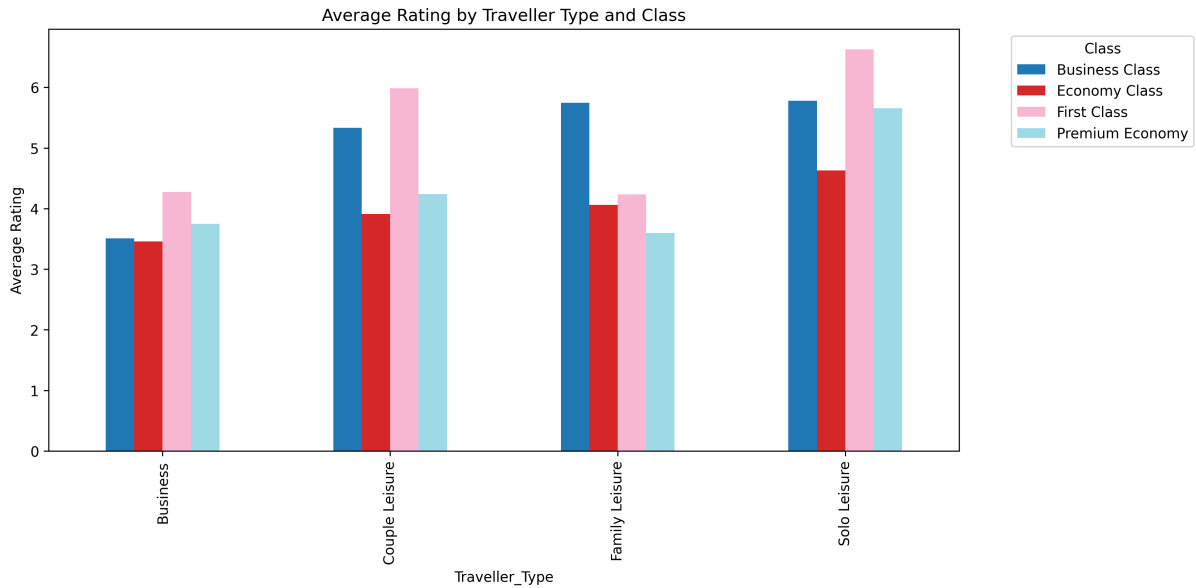


Figure 4: Average rating by traveler type and class (visualization from notebook).

4 Feature Engineering and Selection

After our initial exploration, we moved on to feature engineering to prepare the data for our predictive model. The entire process is implemented in `src/data/pre_process.py` and `src/data/process.py`.

4.1 Passenger Data Processing

For the passenger-related data, we performed the following steps, as implemented in `src/data/pre_process.py`:

- **Class Encoding:** We used dense encoding for the 'Class' column, mapping the different classes to numerical values. This was done because there is an inherent order to the classes (e.g., First Class $\leq$ Business Class $\leq$ Economy Class), and a simple numerical encoding can capture this ordinal relationship.
- **Traveller Type Encoding:** We used sparse (one-hot) encoding for the 'Traveller_Type' column. This creates a new binary column for each traveler type, which is appropriate for categorical features where there is no inherent order between the categories.
- **Verified Encoding:** We used sparse encoding for the 'Verified' column to convert the boolean 'Verified' status into a numerical format suitable for the model.
- **Passenger Name Encoding:** We used frequency encoding for the 'Passenger_Name'. This technique captures the popularity of a name, which might be a proxy for a reviewer's experience or behavior (e.g., frequent flyers might have different rating patterns). It's a way to extract a signal from a high-cardinality categorical feature without creating too many new columns.
- **Sentiment Analysis:** We performed sentiment analysis on the 'Review_content' to extract a compound sentiment score. The raw text of a review is highly unstructured, and sentiment analysis allows us to distill this text into a single, powerful feature that quantifies the reviewer's overall sentiment.

4.2 Spatial Data Processing

To process the spatial data, we first enriched our dataset with external airport information from OurAirports. We then performed the following steps:

- **Coordinate Mapping:** We mapped the start and end coordinates from our dataset to the nearest airports from the external data. We used a BallTree algorithm for this, which is a very efficient way to query for nearest neighbors in a geospatial context.

- **Layover Encoding:** We converted the 'Layover_Route' column into a binary 'Has_Layover' feature. This simplifies the feature by just indicating the presence or absence of a layover, which is often a more impactful signal than the specifics of the layover route.
- **Hierarchical Encoding:** For categorical variables like continent, country, and airport codes, we chose a hierarchical encoding scheme. A simple one-hot encoding would have resulted in a massive increase in dimensionality (an "explosion"), given the large number of unique airports. On the other hand, a simple frequency encoding would have been too destructive, losing valuable information about the spatial relationships between airports. Our hybrid approach, which combines frequency encoding at the country and airport level with one-hot encoding for continents, allowed us to capture the spatial hierarchy and importance of different locations without making the feature space unmanageably large.

4.3 Final Feature Set

The final set of features was then scaled using `StandardScaler` for numerical features, while binary features were passed through. The final list of features used for modeling is:

```
[ 'scaler__Passanger_Name', 'scaler__Class_Encoded', 'scaler__Sentiment_Compound',
  'scaler__Start_Country_freq', 'scaler__End_Country_freq', 'scaler__Start_Code_freq',
  'scaler__End_Code_freq', 'passthrough__Traveller_Business', 'passthrough__Traveller_Couple_Leisure',
  'passthrough__Traveller_Family_Leisure', 'passthrough__Traveller_Solo_Leisure',
  'passthrough__Traveller_Unknown', 'passthrough__Traveller_Various', 'passthrough__Verified_Trip_Verified',
  'passthrough__Has_Layover', 'passthrough__Start_Cont_AS', 'passthrough__Start_Cont_EU',
  'passthrough__Start_Cont_OC', 'passthrough__Start_Cont_SA', 'passthrough__Start_Cont_nan',
  'passthrough__End_Cont_AS', 'passthrough__End_Cont_EU', 'passthrough__End_Cont_OC',
  'passthrough__End_Cont_SA', 'passthrough__End_Cont_nan']
```

5 Predictive Modeling

We implemented and evaluated several classification models to predict our target variable (customer satisfaction). The main script for this is `src/main.py`.

5.1 Models

We experimented with the following models:

- **Multi-Layer Perceptron (MLP):** A feedforward neural network capable of learning complex, non-linear relationships in the data. We chose it for its ability to potentially uncover intricate patterns that other models might miss. It requires feature scaling.
- **Logistic Regression:** A linear model that is simple, fast, and highly interpretable. We used it to establish a performance baseline and to understand the linear separability of our data. It also requires feature scaling.
- **Random Forest:** An ensemble model that builds multiple decision trees. It's robust, less prone to overfitting than single decision trees, and serves as a strong baseline. It does not require feature scaling.
- **Gradient Boosting:** An ensemble model that builds decision trees sequentially, with each tree correcting the errors of the previous one. It's known for its high performance on tabular data and its ability to capture complex feature interactions. It also does not require feature scaling.
- **Support Vector Machine (SVM):** A powerful classifier that finds an optimal hyperplane to separate classes. With a non-linear kernel, it can model complex relationships and is effective in high-dimensional spaces. It requires feature scaling.

5.2 MLP Architecture

Our primary model was an MLP with a carefully chosen architecture to balance performance and prevent overfitting. It consists of an input layer, followed by two hidden layers with 128 and 64 units respectively, and a final output layer. We used the ReLU (Rectified Linear Unit) activation function in the hidden layers, as it is a common and effective choice for deep learning models. To prevent overfitting, we employed two regularization techniques. We added L2 regularization to the dense layers, which penalizes large weights and encourages the model to learn simpler patterns. We also included Dropout layers with a rate of 0.5 after each hidden layer. Dropout randomly sets a fraction of neuron activations to zero during training, which forces the network to learn more robust and redundant representations, further reducing the risk of overfitting. The output layer has a single unit with a Sigmoid activation function, which is appropriate for binary classification problems as it squashes the output to a probability between 0 and 1.

5.3 Evaluation

We evaluated all models on a held-out test set and through cross-validation (where applicable), using **Accuracy** and **Weighted F1-score** as our primary metrics. These metrics provide a balanced view of overall performance, taking both precision and recall into account. The results are summarized in Table ??.

Model	Test Accuracy	Test F1 (Weighted)
Multi-Layer Perceptron (MLP)	0.8127	0.8126
Logistic Regression	0.8000	0.7996
Random Forest	0.8059	0.8059
Gradient Boosting	0.8137	0.8136
Support Vector Machine (SVM)	0.7961	0.7957

Table 1: Model performance on the test set.

Model	CV Accuracy	CV F1 (Weighted)
Multi-Layer Perceptron (MLP)	N/A	N/A
Logistic Regression	0.8075	0.8071
Random Forest	0.8025	0.8025
Gradient Boosting	0.8138	0.8137
Support Vector Machine (SVM)	0.7941	0.7937

Table 2: Model performance under cross-validation.

Across all models, performance clustered tightly around the 80–81% range, demonstrating a consistent and reliable predictive capability across diverse algorithmic approaches. Both ensemble methods and neural architectures performed particularly well, indicating that the features extracted through our engineered pipeline carry strong predictive signals.

The **Gradient Boosting** model achieved the highest overall performance, with a test accuracy of 81.37% and a nearly identical cross-validation score, showcasing excellent generalization. Its ability to iteratively correct residual errors makes it particularly adept at modeling subtle feature interactions in our dataset. The **MLP** closely followed, with an almost identical performance (81.27%), reinforcing that our feature scaling and regularization choices successfully stabilized neural learning.

The **Random Forest** model performed robustly, scoring 80.59% accuracy, further confirming that ensemble-based methods capture non-linear relationships effectively. Meanwhile, **Logistic Regression** despite its simplicity, delivered an impressive 80% accuracy, making it an excellent lightweight baseline for deployment scenarios requiring interpretability and speed. Finally, the **SVM** achieved slightly lower performance at 79.6%, which can be attributed to the high-dimensional nature of our engineered feature space and the model’s sensitivity to hyperparameter tuning.

To gain a deeper understanding of the models’ behavior, we analyzed their confusion matrices. For instance, the MLP correctly classified the majority of satisfied and dissatisfied customers, though it tended to slightly underpredict dissatisfaction (see below). The Gradient Boosting model exhibited a near-perfect balance between the two classes, reinforcing its position as the top-performing model.

Overall, the evaluation confirms that the predictive performance across all models is strong and consistent, with Gradient Boosting and MLP leading the pack. Given their comparable results, the choice between them can be guided by deployment constraints favoring Gradient Boosting for tabular interpretability and MLP for scalability within deep learning pipelines.

6 Model Explainability (XAI)

To gain a deeper understanding of how our predictive model (MLP) arrives at its decisions, we employed two complementary interpretability frameworks: **SHAP** (SHapley Additive exPlanations) for both global and local interpretability, and **LIME** (Local Interpretable Model-agnostic Explanations) for instance-level insights. These tools allow us to open the “black box” of the model and observe how individual features influence its predictions.

6.1 SHAP Analysis

SHAP provides a unified measure of feature importance based on Shapley values from cooperative game theory. The SHAP summary plot in Figure ?? visualizes the overall contribution of each feature across all samples.

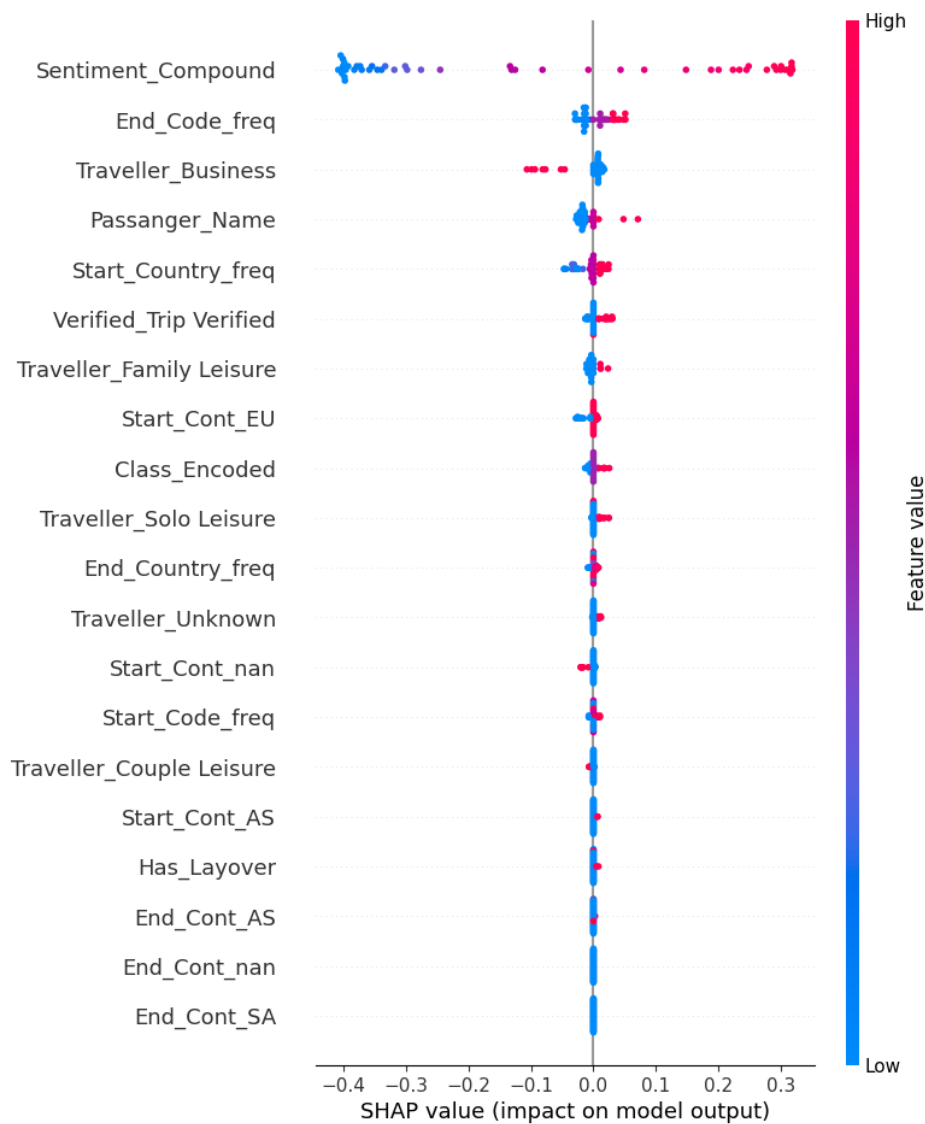


Figure 5: Global SHAP summary plot showing the impact of features on model output.

From the global summary, we observe that **Sentiment_Compound** is the most influential feature in determining satisfaction, with higher sentiment values (in red) pushing predictions toward a positive outcome (satisfied customers). This aligns with our intuition-customers expressing more positive sentiment are more likely to be satisfied.

Other highly impactful features include **End_Code_freq**, **Traveller_Business**, **Passanger_Name**, and **Start_Country_freq**. These variables capture frequent patterns in destination codes, traveler demographics, and origin regions-factors that together shape satisfaction outcomes. Notably, the relatively strong contribution of **Traveller_Business** suggests that business travelers exhibit distinct satisfaction dynamics compared to leisure travelers, while spatial features such as **Start_Country_freq** and **End_Code_freq** highlight regional variations in service experience.

We also examined an individual prediction using a SHAP force plot (Figure ??). The plot shows how each feature pushes the model output higher (toward satisfaction) or lower (toward dissatisfaction) from the baseline probability.



Figure 6: SHAP force plot illustrating feature contributions for a single prediction.

In this specific instance, features such as a low **Start_Country_freq** and negative **End_Code_freq** exerted downward pressure on the predicted satisfaction score, indicating that less common travel routes are associated with lower satisfaction probabilities. Conversely, some encoded spatial or traveler-type attributes partially offset this effect, revealing a nuanced interplay between travel frequency, geography, and traveler identity.

6.2 LIME Analysis

While SHAP provides a consistent global view of feature importance, LIME complements it by offering a highly interpretable local explanation for a single prediction. The LIME report (Figure ??) highlights the top contributing factors for one passenger's predicted satisfaction outcome.

According to LIME, the model predicted this passenger as **satisfied** (class_1) with a probability of **0.82**. The most influential positive driver was a high **Sentiment_Compound** score (0.94), reinforcing the strong link between expressed sentiment and satisfaction. Additionally, a high **Start_Country_freq** and **Start_Cont_EU** contributed positively, implying that flights originating from frequently observed or European regions correlate with better satisfaction levels.

Conversely, features such as a low **End_Code_freq** and negative **Passanger_Name** encoding slightly reduced the satisfaction probability, suggesting that less frequent routes or rare traveler identities may introduce uncertainty or inconsistency in the experience.

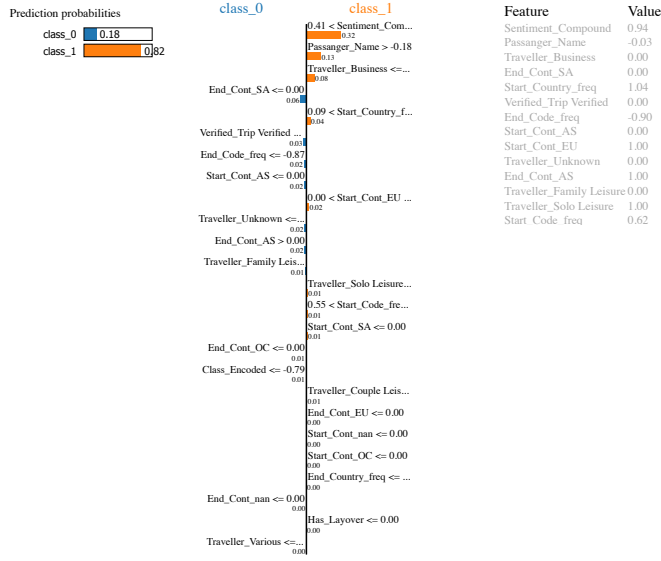


Figure 7: LIME explanation showing feature contributions for one instance.

Overall, the SHAP and LIME analyses converge on the same narrative: **sentiment, route frequency, and traveler type** are the dominant predictors of satisfaction. Together, these explainability tools provide confidence that the model not only achieves high performance but also aligns with real-world intuition about customer behavior.

7 Conclusion

In this project, we have successfully built a pipeline for predicting flight satisfaction. We started with data exploration, moved to feature engineering, and then trained and evaluated several models. Our MLP model showed the best performance, and we used XAI techniques to interpret its predictions. This report provides a comprehensive overview of our work.